

Using the HMRC VAT REST connection.

Author Ian H
Updated 28/01/21
version 1.2

Introduction

Some information about the HMRC REST Client and VAT client components for the HMRC MTD processes. This document should be bundled with the demo projects and installation packages and relates to v1.0.1 of the HmrcRestClient dated 28/01/21. It will try to help explain the classes and how to use them in a real application.

Limitations.

Testing has been done on the HMRC test API as it was at the time of the development and any changes to the API that may have occurred since then and any differences between the test API and the production API are beyond our control.

Preparation.

The starting point is the HMRC documentation on MTD and the REST API, which can/could be found here and does not require a developer account in order to access it:

<https://developer.service.hmrc.gov.uk/api-documentation>

This is not a tutorial on the HMRC process, so will not go into detail on any of that, but will assume that all necessary preparation has been done.

Cut the crap, how does it work?

Assuming the whole HMRC VAT package exists, it needs the HmrcRestClient.pas, HmrcRestSupport.pas, HmrcVatSupport.pas and HmrcVatClient.pas files from CmnUnits.

Add these files to the project and create an HmrcVatClient component. Set the parameters and execute the methods. To help set the anti-fraud headers, there are some helper units in the same folder. The demo projects show how to set the necessary properties and call the various methods. Some is explained below, if you keep reading (recommended).

More detail

The first versions of the component and supporting information were created and released in January 2019. This second version of this document was updated for the release in December 2020 and this version is for the January 21 release.

In the examples, the FMX demo uses a rest client client created in code and targets all test API endpoints available at the time. One VCL demo uses components created in code and the other uses the Hmrc rest component installed on the component tool bar. Both demos only target a few endpoints. Opening the VCL demo which uses the installed component will probably result in some warnings, if the component is not installed. Either way the process of using the component is the same.

It is not necessary to install the component into the palette, but it is an option for those who like setting the properties at design time. Note that it is not possible to install both VCL and FMX

components together., because the components have the same name.

The base unit is HmrcRestClient.pas and this unit has the base HmrcRestClient class. The base class, HmrcRestClient, handles all of the authorisation and authentication, but is not intended to be implemented in code. The authorisation process pops up a built-in form which is either a VCL form or an FMX form. The use of these forms makes the components framework specific and so there is a conditional define of “FMX”, which should be set in any FMX project using the rest components..

The HmrcTestClient class just implements methods to target the endpoints that relate only to tests in the sandbox / test API. The HMRCTestClient class has been moved to its own unit, which does not need to be included in any live application.

The VAT client inherits from the RestClient and adds methods to target each of the VAT endpoints. There are also parallel PAYE clients in a separate branch of the process.

There are some common support units, principally HmrcRestSupport.pas, which is required. It is basically just has a list of constants, classes and functions to help make the process work. It contains the access tokens list. VAT class definitions and some methods to convert json data into vat objects are in HmrcVatSupport.pas.

There are some other helper units, VAT.Headers.Utills.pas, Systematic.FMX.MacAddress.pas, Systematic.AppVersion and uSMBIOS with methods to get/set some of the anti-fraud headers. These may be useful, but are not required to make it work. Some of the methods in these units require the JCL to be installed, can be commented out if that is not available. Adding these units to a VCL project will result in a warning about potential corruption, as there are links to the FMX framework. In testing there have been no actual problems.

There are two similar versions of the process to actually set the headers, in Systematic.VAT.Headers.pas and HmrcHeaders.pas, both of which call on the units above. The user can choose either of these or role their own. The demos use HmrcHeaders, but the approach would be the same for both.

Authorisation and hence Access Tokens are granted in the context of a scope. For VAT the available scopes are **read:vat** and **write:vat**, so 2 sets of tokens are required. Calling the methods in the VatClient will automatically set the scope for that operation, but remember to set the scopes correctly at the start, before authentication.

The TEST component methods.

```
// Check a VRN – accepts a VRN to check and optionally the VRN of the caller
function CheckVRN(Const aValue: string): integer; overload;
function CheckVRN(Const aValue, aCaller: string): integer; overload;
```

The VAT component methods.

```
Function GetLiabilities(Const FromDate, ToDate: TDateTime; Var ACorrelationId: String):
integer; overload;
Function GetLiabilities(Const FromDate, ToDate: TDateTime): TObjectList; overload;
// search obligations for a date range
Function GetObligations(Const FromDate, ToDate: TDateTime; Var ACorrelationId: String;
aStatus: String = "): integer; overload;
Function GetObligations(Const FromDate, ToDate: TDateTime; aStatus: String = "): TObjectList;
```

```

overload;
// search payments for a date range
Function GetPayments(Const FromDate, ToDate: TDateTime; Var ACorrelationId: String):
integer; overload;
Function GetPayments(Const FromDate, ToDate: TDateTime): TObjectList; overload;
// do a returns check for a vat period
Function GetReturn(Const aPeriod: String; Var ACorrelationId: String): integer; overload;
Function GetReturn(Const aPeriod: String): TVatReturn; overload;
// submit a return
Function SubmitReturn(Const aPeriod: String; Const Values: TStringList;
    Const IzFinal: boolean; Var Confirm: String): integer; overload;
Function SubmitReturn(aVatObj: TVatSubmission): integer; overload;
// override to apply some kind of validation
Function SetHmrcID(Const Value: String): integer; Override;

```

All methods return an integer value as one of:

```

RESULT_NONE = 0;
RESULT_OK   = 1;
RESULT_FAIL = -1;
RESULT_NODATA = -2;
RESULT_ERROR = -3;

```

A result of Result_None would in fact indicate an error and should not happen. NoData indicates that the process worked, but no data was returned. It seems weird to traditional VCL database programmers, but in the web world, it seems normal for an empty dataset to be returned as 404 Not Found, which is an error condition. The Liabilities, Obligations and Payments endpoints return an identifiable message, which the process checks for, when receiving 404 messages. The application can then handle NoData in a more appropriate way. Unfortunately the GetReturn method does not get this specific message, just 404 Not Found, so it is not possible to pick out the empty dataset condition.

If not OK, the VAT component provides the return code as **LastCode** (integer) and the error message as **LastMsg** (string). The JSON returned by the endpoint is available as **LastError** (string).

If the result is OK, then the JSON returned is available as **LastValue** (TJSONValue). This requires System.JSON in the uses clause. This is JSON, but it is not necessary to try to handle it. The Submission methods do automatically parse the JSON, either into a string or the VAT object. The JSON from the other methods can be transferred into a corresponding data object by calling the appropriate parse method. See the demos for examples of how to do this.

Using the demos.

The demos only target the Test API, which unfortunately has been shown to be less than perfect at times. Some functions only work with the Test API, but the VAT end points relate to both test and live.

Check list

- Registered user and got a Client ID and Client Secret
- Registered an application and subscribed it to services on the test api. Got an application id.
- Set a callback url.

Some HMRC MTD information and support page links are included at the end.

Instructions for creating a test application are provided in the RegisterAPIApp.doc, which should be included in this package. This was created at the time of the original component development and has screen shots of the HMRC pages as they were at that time. The details shown relate to a test application used at the time. Any keys shown were only valid for the test API and expired long ago. Some of these pages changed on the live HMRC website in December 2020, so follow the links as they are presented.

In order for the demos to work, they need the keys, Client ID/Secret created for the app. These should be stored in an ini file, as shown in the HmrcDevTest.ini example provided in the package Data folder. You can just replace the old demo values, as they will not work and are just included as an example. The projects will show a file dialog on start up, to allow selection of the key file.

Previously there was also a Server Token, used for application level authorisation during testing. This is no longer required and not available for new applications.

For live applications, the keys can be stored anywhere and loaded by any appropriate method. Using the installed component, they can be set at design time. This is entirely up to the application developer.

Some of the demos load access tokens from and/or save access tokens to an ini file, CurrentTokens.ini. The use of an ini file is again just for simplicity and any storage mechanism can be used, for example the Registry or in a database. Loading the tokens is handled by the application, not the VAT component. For saving tokens, the VAT component exposes a THmrcTokenEvent as OnTokenChange and the demos pass in a method that saves to the ini file. Using this event allows automatic saving of new and updated tokens.

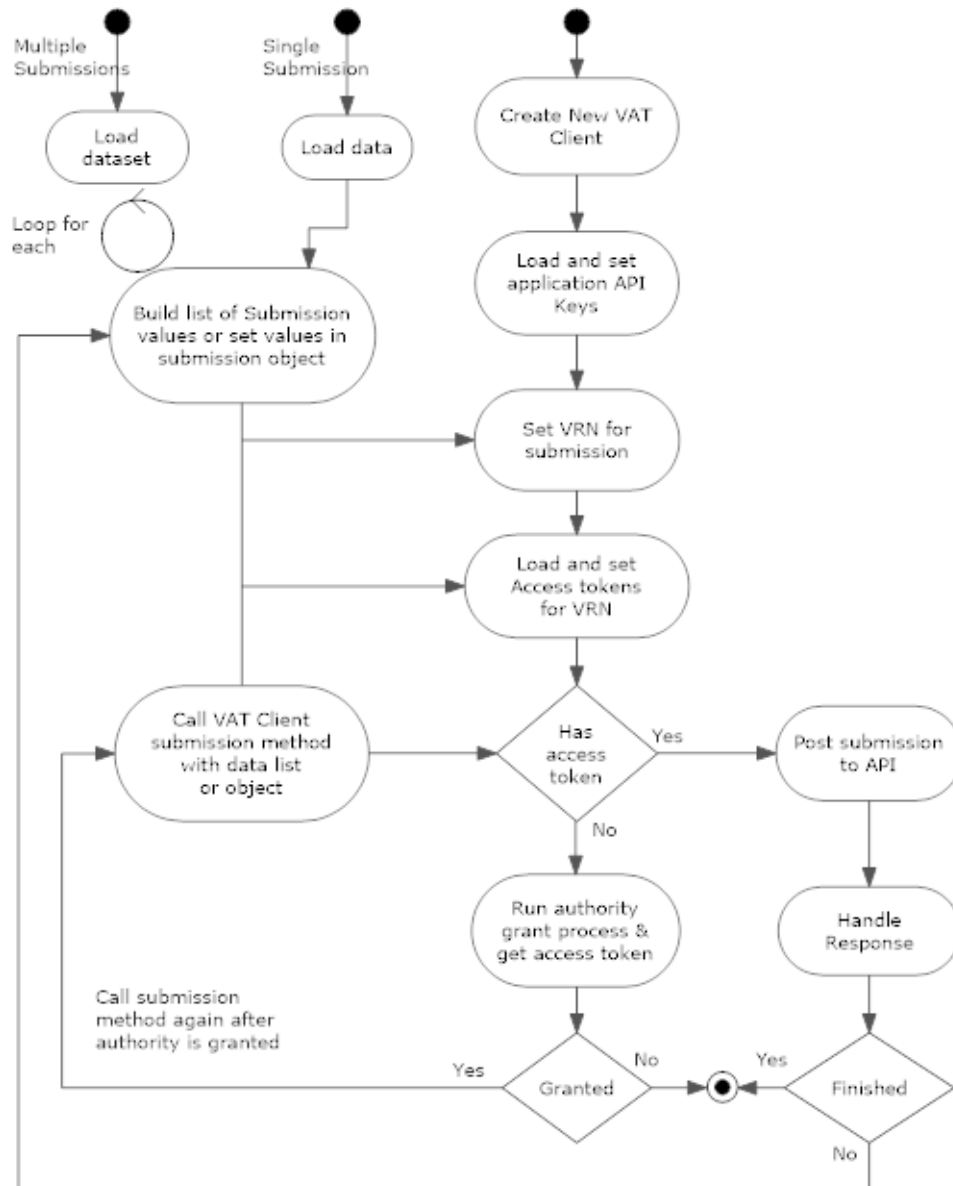
In most cases it will probably be used for only 1 account at a time, so only 1 VRN (UID), with either 1 or both scopes. This would create either 1 or 2 access token objects in the access tokens list. In this case it does not matter whether token details are set using either the AddaToken or SetaToken methods of the VatClient. The difference becomes effective where more than 1 VRN is involved, in which case AddaToken will add a new token for each case, while SetaToken will just update the existing token of the current scope.

When running through a list of accounts, if each is to be used only once, it would be recommended that SetaToken is used for the token details, so that only 1 token for each scope will be created. In this case, the most efficient way to use the process would be to set the new token values before setting the new UID value. This will prevent the current token object being set to nil, because the new UID does not match the access token UID.

The current access token can be reloaded with the ReloadToken method, which by default will check for both UID and scope, but can load for just scope.

The security headers have been a moving target, having been gradually added over the first year of operation, but are now a requirement. They are required for the VAT endpoints and the test fraud headers endpoint.

VAT Submission Process



The FMX demo project.

The FMX demo starts with a main form that has 4 options: Test Client, Vat Fresh, Vat Saved and VRN Checks. Each pops up a new form. The Test Client form has 8 buttons, 3 to test the “hello” end points, 1 to test the available services and 3 to test each of the create services. The last is to test the anti-fraud headers. Check the code to see how each works.

Each call in the demos writes the JSON returned from the endpoint to the screen. It is not necessary for the developer to read/write or even see JSON when calling the VAT API, as the components can handle all of that.

Creating test agents, companies and people generates a new instance in the HMRC test system, which will be available for testing. The details of the new instance will be added to the Users.txt file in the same folder as the key file.

To test the Hello User end point will require the UserId and Password of one of the instances

created above. This can be read from the Users.txt file. The access tokens generated will be saved to the CurrentTokens.ini file in the same folder.

The anti-fraud headers were current at the time of writing, but may be subject to change. The headers are added from within the test form and are the responsibility of the application, not the HMRC REST components. The headers are different for each type of application and the ones shown are for normal desktop apps. It is the responsibility of the application programmer to ensure that the correct headers are used and to keep up to date with any changes that HMRC may make to the requirements. The demos show something that worked on the development machine at the time and with data manipulated for the test, but it may be necessary/advisable to do it differently in other situations.

The 2 VAT forms are just variations on the same theme, where one will look for and use saved access tokens and the other will always request fresh tokens. Using the saved tokens form, if there are no tokens, then it will also go through the validation process to request tokens, but will then save them for future use.

When either form request a token, it will display a message in the memo saying that it is necessary to click the button again. When it has to request the token, it does not loop straight back into the request/submission, but it will remember the token for the lifetime of the form.

(Pounds and pence)		(pounds)	
Due on Sales	1234.56	Sales ex-VAT	10
Due on Aquisitions	78.90	Purchases ex-VAT	20
Total VAT Due	1313.46	Goods Supplied ex-VAT	30
Reclaimed - Current Period	43.21	Aquisitions ex-VAT	40
Net VAT Due	1270.25	Finalised	True

Both forms target all of the VAT endpoints, except the VRN check and show the kind of code necessary for this, but the calls are slightly different, so you are advised to look at both. The User

ID required is the VRN for one of the test companies that has been created with VAT details. It will also require the UserId and Password for the authorisation process if that is necessary and remember that authorisation is required twice, once for read and once for write.

Liabilities, obligations and payments may require the date ranges. Obligations may also require the status value. There was some confusion around the Obligations endpoint and it is the responsibility of the application programmer to keep abreast of any changes that may be made to any of the endpoints.

Viewing returns requires the view period to be set, but does not require dates. The VAT Saved form example converts the json into a vat object and then writes the object values to the memo, because there is nothing else to do with it. All of these processes also return the correlation id as a var parameter. This is probably of little interest, but it is there anyway.

There are edit boxes to enter dummy values for test vat submissions. The VAT component now has 2 methods to handle the submission of VAT Returns. The VAT Fresh form uses the original method and test, where the values are added to a string list are then passed to the vat component. The VAT Saved form uses the VATSubmission object to pass both values in and responses out.

Note that 5 values are pounds and pence and the other 4 are whole pounds. The VAT client component does not validate the data supplied, just passes it on to the endpoint, so it is up to the application developers to ensure that data is supplied in an appropriate format. Submitting decimal values for the values that should be integers will result in an error response.

The responses from the test API are in json format and are displayed in the memo as json; the json string can be accessed as "LastValue". The VAT component returns the data in a string list and the demo then writes this to the memo.

The submit processes also have the ability to save the response to a log file. All that is required to enable this is to set the log file path in the "StoreFolder" property.

The CheckVRN methods (in the test client) call the check vat number endpoint, which is comparatively new and listed separately on the HMRC website. It has 2 formats, one taking just the target VRN to be checked and the other accepting the callers VRN as a second parameter.

A call to check the VRN.

```
procedure TfrmVrnCheck.btnCheckVRNClick(Sender: TObject);
begin
  mmoResponse.Lines.Add("");
  if (TestClient.CheckVrn(Trim(edtTarget.Text)) = RESULT_OK) then
  begin
    // display the response as json
    mmoResponse.Lines.Add(TestClient.LastValue.ToString);
    // or do something else
    GetVRNObject(TestClient.LastValue);
  end
  else begin
    mmoResponse.Lines.Add('Error ' + IntToStr(TestClient.LastCode) + ' ' + TestClient.LastMsg);
    mmoResponse.Lines.Add(TestClient.LastError);
  end;
end;
```

Here the successful responses are initially written to a TMemo and then converted to a VRNObject, but could be handled in any way desired. If it failed, then the error messages are written to the memo instead.

This method here is taken from the FMX demo project, where the GetVRNObject method is :

```
procedure TfrmVrnCheck.GetVRNObject(aValue: TJSONValue);
var
  obj: TVRNResponse;
begin
  obj := HmrcVatSupport.ParseVRNResponse(aValue);
  if Assigned(obj) then
  try
    mmoResponse.Lines.Add("");
    mmoResponse.Lines.Add('VRN response data');
    mmoResponse.Lines.Add(' Name: ' + obj.OrgName);
    mmoResponse.Lines.Add(' Vat No: ' + obj.VatNo);
    mmoResponse.Lines.Add(' Address: ' + obj.Address);
    mmoResponse.Lines.Add(' Postcode: ' + obj.Postcode);
    mmoResponse.Lines.Add(' Country: ' + obj.Country);
  finally
    obj.Free;
  end;
end;
```

This method simply gets the json data converted into a VRN object and then displays the values in the memo, because there is nothing else to do with the data in the demo. The VRN class and other VAT related data classes and conversion methods are defined in the VatSupport unit.

The VCL demos.

There are 2 VCL demos, one using the installed component and the other creating the component in code. The installed component demo, VclRestTest, has a simple form to test the common MTD hello endpoints and a VAT form then tests the VAT Obligations endpoint. The point of this demo is to show the use of the rest clients as installed components.

The other VCL demo, VclVatTest, tests the VRN checking endpoints and a couple of the VAT endpoints. There is nothing in this that is not also in the FMX demo.

How the authentication is handled.

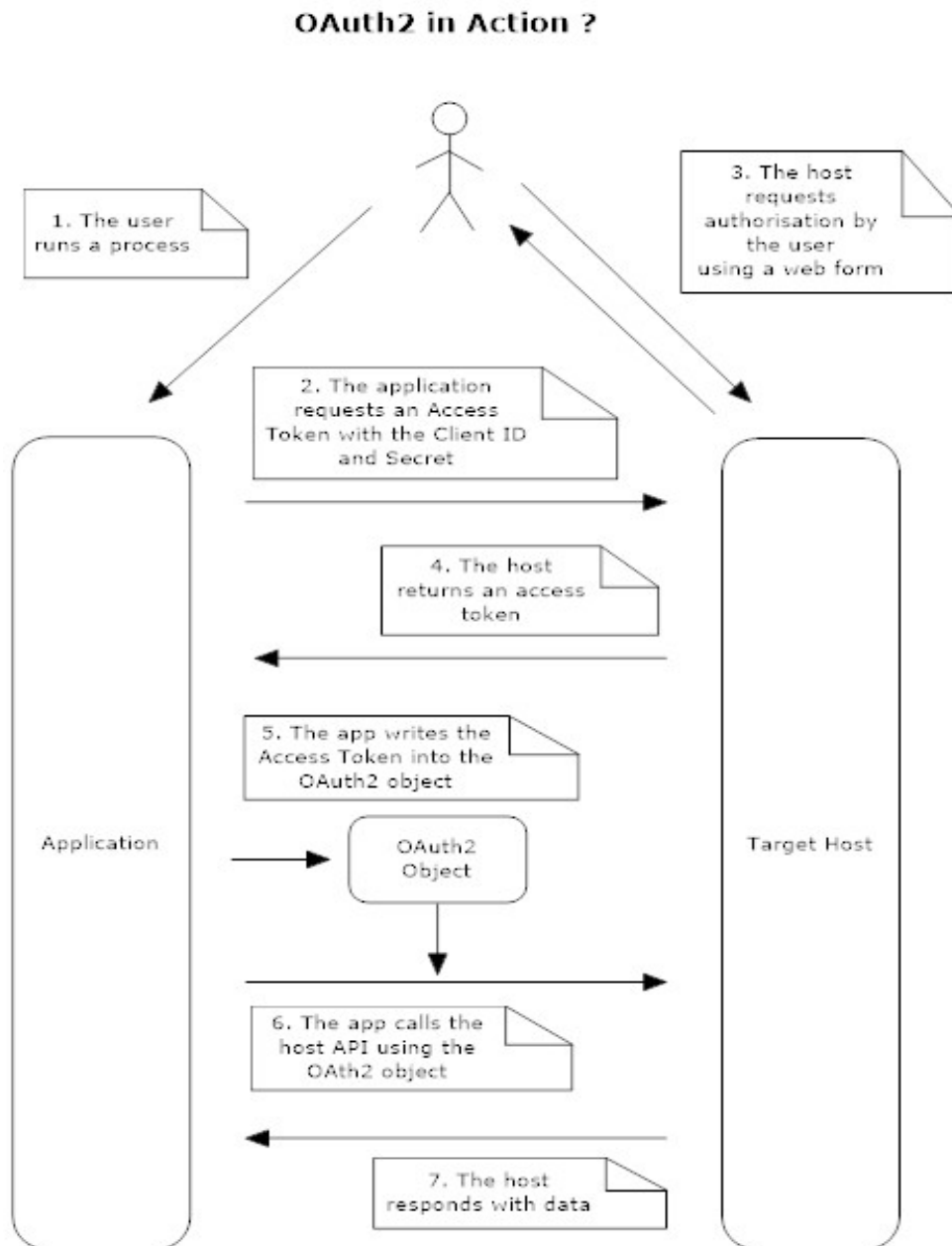
Assuming that the application has been registered and subscribed to the APIs...the test application will try to load the Client Id and Client Secret from an ini file, along with callback url and port for use with the OAuth2 process. The ones supplied with the demo expired a long time ago and will not work now, so please use your current settings.

NB AppKey=ClientId & AppSecret=ClientSecret.

The ServerToken is used for application level logins, which do require the presence of the OAuth2 object, but do not pop up the permissions form. It was created along with the Client Id and Secret when the application was registered. It is used for the Hello Application and New Agent/Company/Person processes.

Using OAuth2 for accessing a user-restricted endpoint. The methods are in the New Access Token (NAT) section of the HMRC Client. The process should be started with a call to the public method `NewAccessToken` and will assume that the `ClientId`, etc. have been loaded successfully, which will be checked with the `CheckReady` function anyway. Also remember to set the scope before calling the method.

OAuth2 – how it works, more or less.



It creates the login form, sets the properties (`AfterRedirect`, `OnClose` and `caption`) and then calls `show` with the `BuildUrl` method. `BuildUrl` does exactly that, including the `ClientId`, callback url and scope. The `SetOAuth2` method just sets the properties of the authenticator component from a bunch of constants and variables.

The `TryForToken` method is what will do the work. This is called after the user logs in with their name and password and grants permission. It is called from the login form's `AfterRedirect` event. It should have now have a code, which it uses to get a pair of access tokens. The access token will then be used in all real calls to the API.

The demos show loading saved details and tokens from an ini file. Loading and saving tokens is the responsibility of the application, not the VAT component. While an ini file has been used for the demos, any method could be used, text file, registry or database, for example.

Refresh Tokens.

HMRC noted that there had been much more trouble with Refresh Tokens than they had expected, so it is worth a paragraph here. Authorisation creates both an Access Token and a Refresh Token. From the time of authorisation, the refresh process will remain active for 18 months. After 18 months, it is necessary to go through the authorisation process again. The refresh process can be used as many, or as few times as necessary during that 18 month period.

Authorisation gives an access token valid for 4 hours and the refresh token, valid for up to 18 months. When using a refresh token to get a new access token, a new refresh token is also created, which replaces the refresh token which has just been used. Using the same refresh token a second time will fail. The refresh process is valid for 18 months, but a refresh token is valid for one use only within that period. When updating an access token it is necessary to save both the new access token and the new refresh token.

For example, where a normal process refreshes and saves tokens, but someone then accesses the service from another application/location and has to do the whole authorisation process, because they do not have the current tokens, this will then invalidate the tokens used by the normal process. | The only option then is to repeat the whole authorisation process in the normal app.

HMRC MTD page links.

These were the relevant help/info pages at the time of writing.

The Hello endpoints were here :

<https://developer.service.hmrc.gov.uk/api-documentation/docs/tutorials>

The New User API was here :

<https://developer.service.hmrc.gov.uk/api-documentation/docs/api/service/api-platform-test-user/1.0>

The VAT API definition was here :

<https://developer.service.hmrc.gov.uk/api-documentation/docs/api/service/vat-api/1.0>

Callback Urls are here

<https://developer.service.hmrc.gov.uk/api-documentation/docs/reference-guide#redirect-uris>

Authorisation errors are here. This can be reached from the callback url page.

<https://developer.service.hmrc.gov.uk/api-documentation/docs/authorisation/user-restricted-endpoints#installed-applications>.

Details about the desktop headers are here:

<https://developer.service.hmrc.gov.uk/guides/fraud-prevention/connection-method/desktop-app-direct/>.

Header formats are here:

<https://developer.service.hmrc.gov.uk/guides/fraud-prevention/getting-it-right/#send-data-in-the-correct-format>